大作业4 用 Hash 与 LSH 加速大模型语义复用

大作业 4 · Hash-based Semantic Retrieval · 2026-10-22
姓名：张祺昀 学号：23300200015

1 问题与数据集

历史请求有 N 条、向量维度为 d。逐个查询全部向量需要 O(Nd) 计算。本系统先通过 Exact Hash 判断原文重复，未命中时才由 LSH 生成候选，再计算候选的真实余弦相似度。研究目标是减少检查范围，同时量化遗漏和索引开销。

实验项	固定设置
历史库	BANKING77 官方训练集，精确去重后 N=10,003；77 个意图 [1]
查询	测试集去除原文重叠并去重，固定种子抽取 300 条；另取 100 条历史原文作 Exact 查询
向量	all-MiniLM-L6-v2，384 维；官方 ONNX AVX2 uint8 量化导出，CPU 推理 [2]
预处理	最多 256 tokens；掩码均值池化 + L2 归一化；保存 float32 向量
相关性	$\tau=0.80$；$G(q)=\{i : u\cdot v_i \geq \tau\}$，离线暴力计算全部 Ground Truth

历史顺序及抽样种子为 20261022。类别标签不参与索引或真值构建。

2 两层 Hash 的分工

Exact Hash：对原始 UTF-8 字符串计算 SHA-256，用哈希表保存“摘要 → ID 列表”。命中摘要后逐项比较原文，避免摘要碰撞造成错误命中。保留大小写、标点和空白；普通 Hash 希望不同文本均匀分散，碰撞只会增加比较成本。

Semantic Hash：相近向量有意以较大概率落入同一桶，碰撞成为候选线索。它既可能遗漏相似项，也可能产生无关候选；用多张表减少遗漏，再用真实余弦过滤候选。

3 随机超平面 LSH 与参数依据

对单位向量 v，独立采样 r~N(0,I)，定义 $h_r(v)=1[r\cdot v\ge 0]$。若余弦为 s、夹角为 θ=arccos(s)，随机超平面将两者分开的概率为 θ/π，故同侧概率为 p(s)=1−arccos(s)/π [3]。

每表 AND：$H_t(v)$ 拼接 k 个独立 bit，同桶概率为 $p(s)^k$。跨表 OR：L 张表使用独立超平面，至少相遇一次的概率为$P(s)=1-(1-p(s)^k)^L$。

事先要求：$s_1=0.80$ 处 $P(s_1)\ge 0.95$，并将 $s\le s_0=0.20$ 的低相似度元素进入概率控制在 5% 内。对每个整数 k，取 $L(k)=\text{ceil}(\ln(0.05)/\ln(1-p(0.80)^k))$，再选首个满足远点约束的 k，以限制表数与投影计算。

k	L(k)	P(0.80)	P(0.20)	判断
11	36	95.10%	6.42%	远点约束不满足
12	46	95.20%	4.67%	主配置，实验前固定
13	58	95.14%	3.34%	可行，但增加表数

s_0 是设计参照，不是全部无关文本的分布假设；0.20<s<0.80 的文本可更频繁碰撞。

4 核心算法与复杂度

```
Build(X, texts, k, L):
  Exact = SHA256(text) -> list of matching IDs
  Sample independent R[t,j] ~ Normal(0, I_d)
  For each i, t: insert i into Bucket[t, H_t(X[i])]

Retrieve(q):
  A = Exact.get(SHA256(q), [])
  A = [i in A where texts[i] == q]   # verify collisions
  if A is not empty: return A      # no embedding needed
  u = Normalize(Embed(q))
  C = empty set
  for t = 1 ... L:
    C = C union Bucket[t, H_t(u)]
  return [i in C where Dot(u, X[i]) >= tau]
```

每表存 uint32 桶偏移及 ID 数组，直接定位桶区间；候选合并后去重，随后只读取 X[C]。不限制候选数量，避免额外截断漏检；不同表重复的 ID 只计算一次余弦。

Exact 摘要需 O(|q|)，表定位期望 O(1)，碰撞后另需原文核验。LSH 建索引的投影开销为 O(NLkd)，另有按桶号排序和目录初始化成本。除 Embedding 外，查询为 O(kLd+B log B+|C|d)，B 为 L 个桶未去重的 ID 总数，此实现排序去重。主配置每次新增 552 次 d 维投影，不能只数候选余弦。

当前密集桶目录的空间为 $O(Nd+NL+Lkd+L\cdot 2^k)$，另加 Exact 表与原文。增大 k 虽能缩小桶，却可能扩大目录；生产实现可用稀疏桶字典改变这一开销。

5 三种方法的实测比较

对 300 个非完全重复查询，273 个存在阈值邻居，27 个 G 为空。Macro Recall 仅对非空 G 平均 $|C \cap G|/|G|$；空 G 单列，不记作 100%。Candidate Ratio 对全部 300 个查询平均 $|C|/N$。

方法	Recall	候选比例	余弦次数/次	均时 ms	P95 ms
Brute-force	100.00%	100.00%	10,003.0	1.096	2.041
Exact Hash Only	0.00%	0.00%	0.0	0.020	0.036
Hash + LSH	98.55%	7.90%	790.7	0.724	1.383

主配置 Micro Recall 为 98.56%，候选量 P95 为 1,340.2；全部 273 个非空查询至少找回一个相关项，但不代表找齐。精筛结果恰为 $C \cap G$，相对于余弦阈值无误报；候选中平均约 775.2 个低于阈值。

计算减少不等于同比提速：余弦次数减少 12.65 倍，实测均时加速 1.51 倍。LSH 还需超平面投影、查桶、候选去重和非连续向量读取；优化过的连续暴力扫描也很快，因此应同时报告操作量和实际时间。

Exact Hash Only 在非重复查询上的 Recall 为 0：速度快，但不能完成语义检索。另取 100 个历史原文查询，两种 Hash 方法均 100/100 命中且进行 0 次余弦计算，可跳过 Embedding。

6 Memory Cost 与计时范围

方法	索引 MiB	向量 MiB	合计 MiB
Brute-force	0	14.65	14.65
Exact Hash Only	2.01	0	2.01
Hash + LSH	5.30	14.65	19.95

LSH 数组负载 3.28 MiB：超平面 0.81、桶偏移 0.72、ID 1.76；再加 11,488 字节容器开销及 2.01 MiB Exact 表。计量使用数组 nbytes 与 Python 对象大小，原文公共存储约 1.04 MiB 单列，模型、解释器和构建临时峰值不计入表中。

结论：LSH 保留全部原向量并增加索引，是用额外空间换取少检查数据。Exact Only 无须向量，但也失去语义检索能力，不能用它的低空间证明 LSH 节省存储。

环境：Windows x86-64，Python 3.12、NumPy 2.5.4、OpenBLAS 单线程；预热后每查询测 5 轮，轮换方法顺序。检索计时包含 SHA-256、查桶、去重和精筛，均使用预计算查询向量。ONNX 编码用 4 线程，10,303 条文本的批量编码共 16.42 秒。

7 参数与数据规模的关系

k / L	Macro Recall	候选比例	LSH 数组 MiB
8 / 46	99.99%	39.74%	2.34
12 / 23	88.22%	4.25%	1.64
12 / 46	98.55%	7.90%	3.28
12 / 92	99.90%	15.38%	6.57
16 / 46	86.28%	1.66%	14.33

固定 L，增大 k 会缩小候选但增加漏检；固定 k，增大 L 会提高召回、候选量及存储。k=16 的候选更少，但密集桶目录使空间变大。三个预定随机种子的主配置 Recall 范围为 97.89% 至 98.65%，候选比例为 7.90% 至 8.99%；未择优换种子。

历史条数 N	平均候选数	候选比例	平均无关候选
2,000	156.3	7.82%	153.3
5,000	394.4	7.89%	386.8
10,003	790.7	7.90%	775.2

规模实验使用同一组超平面、同一批 300 个查询及嵌套随机历史子集，真值随子集重算。固定 k、L 时 $E|C(q)|=\Sigma_i P(\mathrm{sim}(q,x_i))$。分布稳定时比例近似不变，但候选绝对量随 N 增长；本实验约 7.9% 并不意味着始终只检查数百条。

若无关项进入概率 $\alpha=0.001$，$N=10^8$ 时仍有约 10^5 个无关候选。若将 $s\le s_0$ 的无关项期望总量限制在 B 内，可由 $P(s_0)\le L\cdot p(s_0)^k$ 得到充分条件 $k\ge \ln(NL/B)/\ln(1/p(s_0))$，并增大 L 维持近点召回；二者须联合求解。对 $s>s_0$ 的文本该界不适用。随规模增大 k、L 时，投影及 NL 存储也上升。

可改用稀疏桶降低目录空间，或多探测 LSH 用更多桶访问换较少的表；高密度桶再划分可降候选，却须重新评估边界漏检。上述为规模扩展方案，未作为已验证改进。固定参数仍有 $\Theta(N)$ 期望候选成本，极端情况下所有向量同桶仍会扫描全库。

8 最后思考

好的 Embedding 解决如何表示语义，却不消除搜索成本。单次扫描仍需 $O(Nd)$，若全部请求两两比较则需 $O(N^2d)$，数据增长会吞噬算力、内存带宽与响应时间。Hash 的价值既包括精确键的快速查找，更在于按相似性组织候选，把昂贵精筛集中到更有希望的区域。设计者必须同时控制漏检、候选数量、投影成本和索引空间，并随规模调整参数。因此语义检索仍是选择数据结构、概率保证与资源预算的算法设计问题。

参考与复现

[1] Casanueva et al. Efficient Intent Detection with Dual Sentence Encoders, 2020. 数据：github.com/PolyAI-LDN/task-specific-datasets（CC BY 4.0）。
[2] Sentence Transformers. huggingface.co/sentence-transformers/all-MiniLM-L6-v2（Apache-2.0）。
[3] Charikar. Similarity Estimation Techniques from Rounding Algorithms. STOC 2002, pp. 380-388. DOI:10.1145/509907.509965。